

INFORME:

Plataforma Micro SaaS - Multi

Tenant para Gestión de Barberías



	Integrante
Rut	21.000.448-3
Nombres	Joel Bastián
Apellidos	Arancibia Hernández
Roles	Lider / Full-Stack Developer

Índice

Introducción.....	3
Descripción Proyecto.....	4
Contexto del proyecto.....	4
Metodología.....	5
Descripción técnica de la solución.....	5
Plan de Trabajo.....	6
Planificación semestral.....	7
Configuración del ambiente de pruebas.....	8
Estrategia de pruebas.....	9
Procedimiento de Copia de Seguridad.....	11
Configuración del servidor Cloud.....	11
Diagramas.....	12

Introducción

Actualmente, muchos negocios del rubro de barberías continúan gestionando sus reservas mediante redes sociales, mensajería instantánea o métodos manuales, lo que puede generar desorganización, pérdida de clientes y dificultades para administrar horarios y servicios.

El presente proyecto tiene como objetivo desarrollar una plataforma web Micro-SaaS Multi-Tenant orientada a la gestión digital de reservas para barberías. La solución permitirá que múltiples negocios utilicen una misma infraestructura tecnológica, manteniendo independencia en sus datos, configuraciones y funcionamiento interno.

La plataforma considera funcionalidades como administración de barberos, servicios, horarios y reservas online, además de herramientas de autenticación, persistencia de datos y comunicación automatizada mediante correo electrónico.

Para el desarrollo de la solución se utilizaron tecnologías modernas enfocadas en aplicaciones web escalables, utilizando Next.js para el frontend, NestJS para el backend, PostgreSQL mediante Supabase como base de datos y Prisma ORM para la gestión y modelado de la información.

Este informe presenta la definición del proyecto, arquitectura propuesta, planificación, configuraciones técnicas, ambiente de pruebas y diagramas utilizados para el diseño y desarrollo de la solución.

Descripción Proyecto

En la descripción debes señalar brevemente el nombre, descripción, objetivos y alcance del proyecto

Tema	Plataforma Micro-SaaS Multi-Tenant para Gestión de Reservas de Barberías.
Descripción	Desarrollo de una plataforma web que permite a barberías gestionar sus reservas de forma digital mediante un sistema de agendamiento online. El sistema funcionará bajo un modelo multi-tenant, permitiendo que múltiples barberías utilicen la misma plataforma, pero cada una con su propia página web personalizada, servicios, barberos, horarios y reservas.
Problemática detectada	Las barberías pequeñas y medianas carecen de canales de venta digitales automatizados, dependiendo muchas veces de agendas manuales o a través de redes sociales, lo que genera problemas como: - Pérdida de tiempo y/o desorden de las reservas - Reservas duplicadas - Pérdida de clientes por falta de organización, etc...
Solución esperada	Un Software como servicio (SaaS) donde cada barbería podrá obtener su propia plataforma de reservas online interactiva con sus propio estilo visual.
Objetivos generales	Desarrollar una plataforma web de reservas que permita a barberías gestionar citas de forma eficiente mediante un sistema digital accesible para clientes y administradores.
Objetivos específicos	Los objetivos específicos de este proyecto son: - Desarrollar un sistema de reservas online para clientes. - Permitir a los administradores gestionar barberos, servicios y horarios. - Implementar un sistema multi-tenant que permita a múltiples barberías usar la misma plataforma con configuraciones independientes. - Facilitar la visualización de disponibilidad de horarios. - Reducir errores de agendamiento y mejorar la organización del negocio.
Alcance del proyecto	El sistema funcionará mediante una arquitectura multi-tenant , donde múltiples barberías utilizarán la misma plataforma, pero cada una tendrá su propio espacio de gestión y configuración. Algunas de las funciones son: - Registro y administración de barberías - Gestión de barberos y servicios - Creación y gestión de reservas - Visualización de horarios disponibles según barbero - Recordatorios automatizados a través de correo electrónico - Administración de reservas por parte de barbero y administrador.

Contexto del proyecto

Especificar los antecedentes y motivación para el desarrollo del proyecto, publico objetivo, estado del arte, origen de la problemática y descripción del problema resolver.

El rubro de las barberías ha experimentado un crecimiento exponencial, pero su madurez tecnológica es baja. La motivación de este proyecto nace de la necesidad real de negocios (como Black & White Studio) de profesionalizar su atención. Este método puede generar problemas como desorganización en los horarios, reservas duplicadas o dificultad para gestionar múltiples barberos y servicios. Existen plataformas de agendamiento en el mercado como **Setmore**, sin embargo estas funcionan como marketplaces donde los barberos deben registrarse dentro de la plataforma y dependen de su infraestructura y marca. La propuesta de este proyecto busca desarrollar una plataforma que permita a cada barbería contar con su propia página web para reservas, manteniendo una experiencia más personalizada para el negocio y sus clientes.

El sistema se basará en un modelo Micro-SaaS multi-tenant, donde múltiples barberías podrán utilizar la misma infraestructura tecnológica, pero manteniendo sus datos separados y configuraciones independientes.

El público objetivo del sistema son:

- Barberías pequeñas y medianas
- Barberos independientes
- Negocios que buscan digitalizar su sistema de reservas

El desarrollo de este proyecto permitirá mejorar la organización del negocio, optimizar la gestión de citas y facilitar la experiencia de reserva para los clientes.

Metodología

Describir la metodología a utilizar para el desarrollo del proyecto y justificar su uso.

Descripción Metodología
Para el desarrollo de este proyecto se utilizará una Metodología Ágil basada en Kanban, adaptada para un desarrollador independiente. Esta metodología permite visualizar el flujo de trabajo mediante un tablero de tareas con distintas etapas: “To do”, “In Progress”, “Done”. Utilizando diferentes herramientas como Trello, Jira, GitHub, etc... Al ser un proyecto individual, se prescinde de las Daily Scrum, enfocándose solamente en el desarrollo iterativo e incremental limitando el “Work In Progress” para así evitar cuellos de botella y mantener el enfoque en terminar un módulo antes de comenzar otro. Como se trata de una arquitectura Micro-SaaS Multi-Tenant, esta metodología facilita la división del sistema en entregas continuas y funcionales. Se construirá el software de forma iterativa: primero la DB, luego la API y finalmente la interfaz para así garantizar que siempre haya una versión estable del producto.

Descripción técnica de la solución

Describir los detalles técnicos del producto a desarrollar

Especificaciones técnicas de la solución a desarrollar
Infraestructura a utilizar: Despliegue distribuido en la nube: Vercel para alojar el frontend, Render para el alojamiento del backend y SupaBase para mantener alojada la base de datos. Arquitectura de software: Será una arquitectura de 3 capas o Three-Tier enfocada en un modelo Micro-SaaS Multi-Tenant, donde tendremos capa de presentación, capa de lógica de negocio y capa de Acceso a datos. Lenguajes de programación: Los lenguajes de programación a utilizar son: - TypeScript, tanto en el frontend como en el backend para el desarrollo de la lógica de la aplicación. - JavaScript, lenguaje base del entorno Node.js sobre el cual se ejecuta la aplicación. - HTML, para estructurar el contenido de la interfaz web. - CSS, para definir el diseño y estilos visuales de la página web Framework, dependencias y/o librerías a utilizar: Entre los frameworks, dependencias o librerías a utilizar, están: - Next.js para el entorno Frontend - NestJS para la API del Backend - Tailwind CSS para diseño de la interfaz de forma rápida y consistente - Prisma ORM para la comunicación y modelado de la base de datos - JWT (JSON Web Tokens) para la seguridad y autenticación del sistema

Base de datos:

Este proyecto utilizará una base de datos relacional PostgreSQL, gestionada mediante la plataforma Supabase. Donde el acceso a esta será gestionado por Prisma ORM, lo que permitirá la ejecución de consultas seguras y eficientes.

Repositorio:

El código fuente del sistema será gestionado mediante Git y alojado en la plataforma GitHub, permitiendo el control de versiones, trabajo colaborativo y seguimiento de cambios durante el desarrollo del proyecto. Existirá 1 repositorio para este proyecto, que contiene tanto el frontend como el backend del proyecto.

Otros:

Para este proyecto, también se utilizará Cloudinary para el almacenamiento y gestión de imágenes y logos. Y también se utilizará Resend para el envío automático de correos electrónicos.

Plan de Trabajo

Define la planificación de tu Proyecto de acuerdo a las evidencias a realizar durante el semestre.

Nro .	Nombre de Actividades/Tareas	Descripción Actividads/Tareas	Recursos	Duración de la actividad	Responsable	Observaciones
1.-	Definición del Proyecto y Análisis de Requisitos	Elaboración del documento de registro. Definición de alcance, objetivos y modelo de datos inicial (MER).	Documento Guía 1, PC	2 semanas	Desarrollador Full-Stack	Entrega clave para la validación inicial del proyecto por parte del docente.
2.-	Configuración de Entorno y Arquitectura Base	Inicialización de Next.js, NestJS y conexión con Supabase y Prisma ORM.	PC, GitHub, VS Code, Node.js, Vercel/Render.	1 semana	Desarrollador Full-Stack	Fundamental estructurar bien las carpetas y commits iniciales en el repositorio.
3.-	Desarrollo Backend (API, Seguridad y BD)	Programación de los endpoints principales (barberos, reservas). Implementación de autenticación con JWT y RLS.	PC, NestJS, Prisma, Postman (para pruebas de API).	4 semanas	Desarrollador Full-Stack	Los endpoints deben ser probados rigurosamente en Postman antes de armar el Front end.
4.-	Desarrollo Frontend e Interfaces (UI)	Creación de las vistas web	PC, Next.js.	5 semanas	Desarrollador Full-Stack	Desarrollar componentes con vista

		(Dashboard, panel, reservas) usando Tailwind CSS, shadcn/ui				móvil (Diseño responsivo)
5.-	Integración y Pruebas de Calidad (QA)	Conexión del Frontend con la API del Backend. Ejecución de pruebas, validación de flujos y corrección de bugs.	PC, Entorno de pruebas local.	2 semanas	Desarrollador Full-Stack	Registrar todos los bugs encontrados y documentar su solución técnica.
6.-	Despliegue a Producción (Deploy)	Subida del código a los servidores en la nube (Vercel para Front y Render para Back). Configuración de Supabase.	Cuentas cloud (Vercel, Render, Supabase).	1 semana	Desarrollador Full-Stack	Revisar minuciosamente que las variables de entorno (.env) estén correctas.
7.-	Documentación Final y Presentación	Cierre del proyecto, preparación de manuales y ensayo de la defensa final.	PC, Canva, Word.	2 semanas	Desarrollador Full-Stack	Preparar un entorno de demostración limpio para la presentación al profesor.

Planificación semestral

Describir todas las tareas a realizar durante el semestre, considerando los hitos de entregas en cada una de las fases.

[illegible]

Desarrollo Backend (API, Seguridad y BD)				X	X	X	X											
Desarrollo Frontend e Interfaces (UI)								X	X	X	X	X						
Integración y Pruebas de Calidad (QA)													X	X				
Despliegue a Producción (Deploy)															X			
Documentación Final y Presentación																X	X	

Configuración del ambiente de pruebas

Para el desarrollo y validación del sistema de reservas se configuró un ambiente de pruebas local que permite ejecutar y verificar el funcionamiento de la solución antes de su despliegue en producción. Este ambiente replica parcialmente la arquitectura proyectada para producción, utilizando las mismas tecnologías, configuraciones y servicios externos definidos para la plataforma.

El objetivo principal del ambiente de pruebas es validar la correcta integración entre frontend, backend y base de datos, además de comprobar el funcionamiento de las funcionalidades principales del sistema de reservas.

La arquitectura del ambiente de pruebas se compone de:

- ☐ Frontend desarrollado con [Next.js](#) y TypeScript
- ☐ Backend desarrollado con NestJS
- ☐ ORM Prisma para la comunicación con la base de datos
- ☐ Base de datos PostgreSQL alojada en Supabase
- ☐ Variables de entorno para configuración de credenciales, JWT y conexiones externas

Configuración del Frontend:

El frontend se ejecuta localmente utilizando un servidor de desarrollo proporcionado por Next.js. Este componente es responsable de la interfaz gráfica del sistema, incluyendo la visualización de barberos, servicios y formularios de reservas.

Tecnologías utilizadas:

- [Next.js](#)
- TypeScript
- Tailwind CSS

Variables de entorno utilizadas:

- NEXT_PUBLIC_API_URL=

Esta variable permite establecer la comunicación entre el frontend y la API del backend.

Configuración del backend:

El backend se ejecuta localmente utilizando NestJS y se encarga de gestionar la lógica de negocio, validaciones, autenticación y comunicación con la base de datos.

Tecnologías utilizadas:

- NestJS
- Prisma ORM
- JWT
- PostgreSQL

Variables de entorno utilizadas:

- DATABASE_URL=
- JWT_SECRET=
- TOKEN_SECRET=
- RESEND_API_KEY=
- FRONTEND_URL=
- CLOUDINARY_CLOUD_NAME=
- CLOUDINARY_API_KEY=
- CLOUDINARY_API_SECRET=

Estas variables permiten configurar la conexión con Supabase, autenticación JWT y servicios externos utilizados por la plataforma.

Configuración de la base de datos:

La base de datos utilizada corresponde a PostgreSQL alojada en Supabase. Esta configuración permite almacenar de manera persistente la información del sistema, incluyendo reservas, servicios, usuarios y barberos.

Para la sincronización entre Prisma y la base de datos se debe ejecutar el siguiente comando.

`npx prisma db push`

Estrategia de pruebas

Para esta entrega se definió una estrategia de pruebas funcionales manuales enfocada en validar el correcto funcionamiento del stack tecnológico utilizado y la integración entre sus componentes principales.

Las pruebas consideran validaciones de interfaz, lógica de negocio y persistencia de datos como se puede ver en la siguiente tabla.

Componente	Tecnología	Tipo de Prueba	Objetivo
Frontend	Next.js	Pruebas funcionales de interfaz	Verificar navegación, formularios y flujo de reservas
Backend	NestJS	Pruebas de API REST	Validar reglas de negocio y procesamiento de reservas
Base de datos	PostgreSQL / Supabase	Pruebas de persistencia	Verificar almacenamiento correcto de información
ORM	Prisma	Pruebas de integración	Validar comunicación entre backend y base de datos
Seguridad	JWT y variables de entorno	Pruebas de configuración	Validar autenticación y manejo seguro de credenciales

Las pruebas funcionales realizadas fueron las siguientes:

	Caso de prueba	Resultado esperado	Resultado obtenido
CP-01	Visualización de barberos disponibles	Mostrar listado correctamente	EXITOSO
CP-02	Selección de servicio	Mostrar formulario de reserva	EXITOSO
CP-03	Registro de reserva válida	Crear reserva en sistema	EXITOSO
CP-04	Validación de campos obligatorios	Mostrar mensaje de error	EXITOSO
CP-05	Persistencia en base de datos	Guardar reserva en PostgreSQL	EXITOSO
CP-06	Comunicación frontend-backend	Respuesta correcta de API REST	EXITOSO

Las pruebas realizadas permitieron verificar el funcionamiento general del sistema y validar la correcta integración entre los distintos componentes del stack tecnológico utilizado.

Procedimiento de Copia de Seguridad

Con el objetivo de resguardar la información crítica del sistema, se definió una estrategia básica de respaldo para la base de datos utilizada durante el desarrollo y pruebas de la plataforma.

Actualmente, la aplicación utiliza una base de datos PostgreSQL alojada en Supabase, permitiendo almacenamiento persistente y administración cloud de la información relacionada con reservas, servicios, usuarios y barberos.

Para futuras etapas del proyecto, se considera mantener un ambiente de pruebas separado del ambiente de producción, permitiendo restaurar copias de seguridad sin afectar los datos reales de la plataforma.

Procedimiento definido:

1. Acceder al panel administrativo de Supabase.
2. Seleccionar el proyecto correspondiente a la plataforma.
3. Acceder al módulo de administración de base de datos PostgreSQL.
4. Generar o exportar un respaldo de la base de datos.
5. Almacenar el archivo de respaldo en un entorno seguro.
6. Restaurar la copia de seguridad en un ambiente de pruebas cuando sea necesario.

La información que se respalda será toda la que contenga la base de datos, desde reservas hasta configuraciones principales del sistema, como por ejemplo la cantidad de horas de anticipación para los recordatorios de las reservas.

Esta estrategia permite también mantener disponibilidad y recuperación de la información frente a posibles fallos o errores durante futuras etapas de desarrollo y despliegue.

Configuración del servidor Cloud

La aplicación fue preparada para un futuro despliegue en infraestructura cloud, utilizando configuraciones compatibles con entornos de producción tanto para frontend como backend.

Actualmente, el sistema se ejecuta en ambiente local para fines de desarrollo y pruebas, utilizando variables de entorno para manejar credenciales, conexiones y configuraciones sensibles de manera segura.

Configuraciones implementadas:

- Conexión remota a base de datos PostgreSQL mediante Supabase.
- Variables de entorno para autenticación JWT.
- Configuración de comunicación entre frontend y backend mediante API REST.
- Configuración de servicios externos como Cloudinary y Resend.
- Estructura preparada para despliegue en plataformas cloud como Vercel y Render.

Diagramas

A continuación se presentan los diagramas utilizados para representar la arquitectura, funcionamiento, y estructura del sistema a desarrollar. Estos diagramas permiten visualizar de manera clara los componentes principales de la solución, el flujo de trabajo del sistema de reservas, la relación entre las entidades y el ciclo de vida del software, facilitando así el análisis, diseño y desarrollo de la plataforma.

Diagrama de Casos de Uso:

El siguiente diagrama representa los distintos actores que interactúan con el sistema y las funcionalidades disponibles para cada uno. Se identifican los procesos principales relacionados con la gestión de reservas, administración de servicios y control de barberos dentro de la plataforma.



Diagrama de Flujo de Trabajo:

El diagrama de flujo representa el funcionamiento del proceso principal del sistema correspondiente a la creación de reservas. Este flujo describe paso a paso la interacción del cliente con la plataforma, desde la selección del barbero y servicio hasta la confirmación o validación de la reserva.

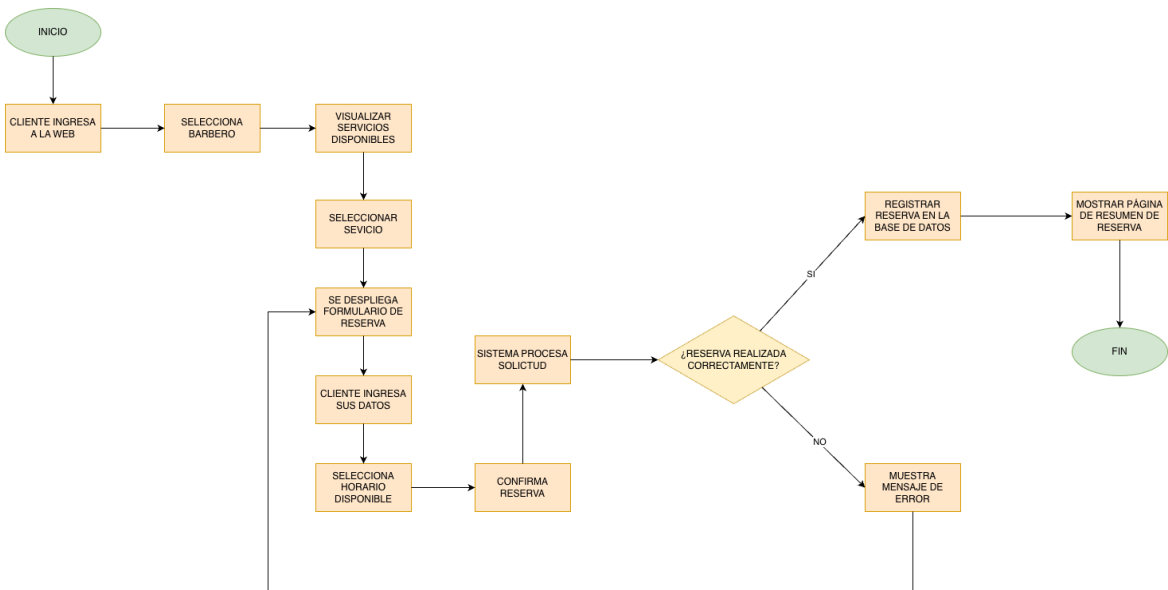


Diagrama de Entidades:

El siguiente diagrama muestra la estructura relacional de la base de datos del sistema. Se representan las entidades principales involucradas en la gestión de reservas, así como sus relaciones y atributos más relevantes para el funcionamiento de la plataforma.

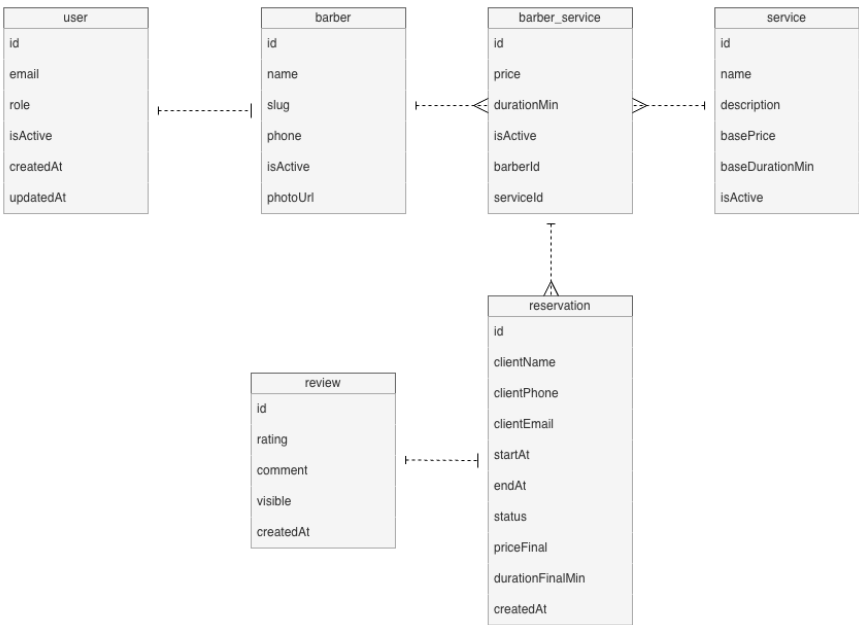


Diagrama de Despliegue:

El diagrama de despliegue representa la arquitectura tecnológica utilizada por el sistema, mostrando la distribución de los componentes principales entre frontend, backend y base de datos. Además, se muestran las tecnologías utilizadas y la comunicación entre cada capa de la aplicación.

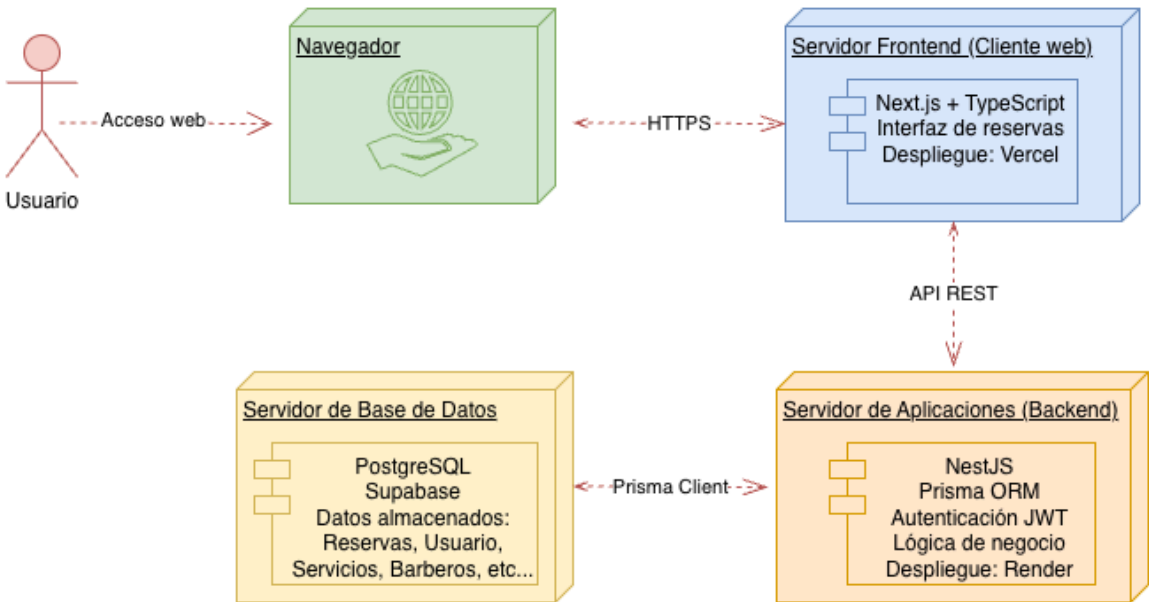


Diagrama Ciclo de Vida del Software:

El siguiente diagrama representa el ciclo de vida utilizado durante el desarrollo del proyecto. Este proceso considera etapas de análisis, diseño, desarrollo, pruebas, despliegue y mantenimiento continuo, permitiendo una evolución iterativa y organizada de la solución de software.

